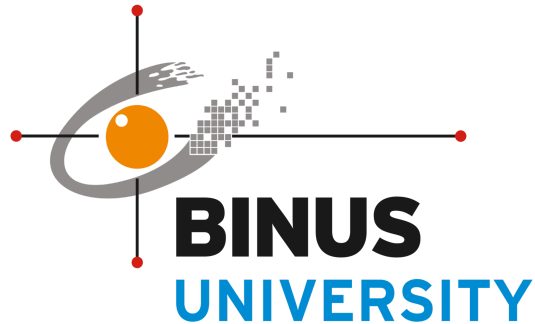


LAPORAN TUGAS KELOMPOK DEEP LEARNING TRAINING MODELS



[Deep Learning]

Dosen pengampu: RICKY REYNARDO SISWANTO, S.T., M.Kom

Kelas: LE01

- Michelle Hioe - 2702357402
- Sefanya Putri Novira - 2702244782
- Fernando Gunawan - 2702247960

BINUS University

Semester Ganjil 2025/2026

DAFTAR ISI

BAB I INTRODUCTION

- 1.1 Latar Belakang
- 1.2 Tujuan
- 1.3 Ruang Lingkup

BAB II DATASET

- 2.1 Data Explanation
- 2.2 Eksplorasi Data (Exploratory Data Analysis)

BAB III MODELLING

- 3.1 Arsitektur Model
- 3.2 Skenario Pelatihan
- 3.3 Diagram Arsitektur

BAB IV EVALUATION

- 4.1 Metrik Evaluasi
- 4.2 Comparison (Perbandingan Model)
 - 4.2.1 Menggunakan CNN (ResNet)
- 4.3 Visualisasi dan Analisis Hasil

BAB V DEPLOYMENT

- 5.1 Aplikasi
 - 5.1.1 Frontend (User Interface)
 - 5.1.2 Backend & Logic
 - 5.1.3 Deep Learning
 - 5.1.4 Hosting
- 5.2 Alur Kerja Sistem
 - 5.2.1 Input Pengguna (Frontend)
 - 5.2.2 Pemrosesan Backend & Model
 - 5.2.3 Integrasi Google Gemini API
 - 5.2.4 Output ke Pengguna

BAB VI REFLECTION

- 6.1 Analisis Kritis
- 6.2 Kendala dan Pembelajaran

REFERENCE

LAMPIRAN

BAB I

INTRODUCTION

1.1. Latar Belakang

Penyakit pada tanaman merupakan salah satu permasalahan utama dalam sektor pertanian karena dapat menurunkan kualitas dan kuantitas hasil panen secara signifikan. Dalam praktiknya, identifikasi penyakit tanaman masih banyak dilakukan secara manual berdasarkan pengamatan visual oleh petani atau tenaga ahli. Metode ini tidak hanya memerlukan pengalaman yang cukup, tetapi juga bersifat subjektif dan kurang efisien, terutama ketika harus menangani jumlah tanaman yang besar.

Saat ini, penyakit menular mengurangi potensi hasil panen rata-rata sebesar 40%, dengan banyak petani di negara berkembang mengalami kerugian hasil panen hingga 100%. Perihal masalah informasi berskala modern, perlu digunakan solusi modern seperti teknologi AI untuk mengatasinya. Begitu banyak data dari kasus penggunaan nyata petani yang mengidentifikasi tanaman sakit hanya dengan melihat daun saja yang dapat digunakan untuk melatih model deep learning agar dapat melakukan hal yang sama dalam skala yang lebih besar.

Perkembangan teknologi computer vision dan Deep Learning membuka peluang untuk melakukan deteksi penyakit tanaman secara otomatis melalui bentuk/tekstur/warna daun. Dengan memanfaatkan model klasifikasi berbasis DL, sistem dapat mempelajari pola visual tertentu yang berkaitan dengan kondisi daun, baik sehat maupun terserang penyakit. Pendekatan ini diharapkan mampu membantu petani dalam melakukan diagnosis awal secara lebih cepat dan konsisten.

Oleh karena itu, penelitian ini diharapkan dapat mengembangkan sebuah sistem deteksi penyakit tanaman berbasis Deep Learning yang tidak hanya memiliki performa klasifikasi yang baik, tetapi juga efisien dan siap diimplementasikan dalam bentuk aplikasi yang mudah digunakan oleh pengguna.

1.2 Tujuan

Tujuan dari eksperimen ini adalah sebagai berikut:

1. Mengembangkan model Deep Learning yang mampu mengidentifikasi penyakit tanaman berdasarkan gambar daun secara akurat.
2. Menerapkan teknik transfer learning untuk meningkatkan performa klasifikasi penyakit tanaman.
3. Membandingkan performa EfficientNet dengan CNN yang menggunakan residual block dalam tugas klasifikasi daun.
4. Membangun aplikasi berbasis website sebagai antarmuka pengguna untuk mengakses model secara mudah dan efisien.

5. Mengintegrasikan model yang telah dilatih ke dalam sistem deployment sehingga dapat digunakan dalam skenario nyata.

1.3 Ruang Lingkup

Proses dimulai dari eksplorasi dan analisis dataset PlantVillage untuk memahami karakteristik data, dilanjutkan dengan tahap preprocessing dan augmentasi data. Selanjutnya, dilakukan perancangan dan pelatihan model Deep Learning menggunakan arsitektur EfficientNet dan CNN dengan residual block, serta evaluasi performa model menggunakan metrik yang sesuai. Tahap akhir dari eksperimen ini adalah deployment model ke dalam aplikasi berbasis website sehingga sistem dapat diakses dan digunakan secara mudah oleh pengguna.

Dataset yang digunakan adalah PlantVillage dataset yang dihost di Kaggle. Eksplorasi data memperlihatkan bahwa distribusi kelas tidak seimbang, kemungkinan lebih banyak gambar daun yang sakit, dan gambar antar kelas mirip secara visual. Arsitektur model yang digunakan adalah EfficientNet, dataset pertama dipisah dengan 80% training data dan 20% validation data. Setelah itu, data di pre-process dengan Z-score normalization per channel RGB dan di augmentasi seperti memutar gambar supaya model lebih bisa menghadapi data baru. Model di train secara 20 epoch dan berhasil mendapatkan metrik x% akurasi.

Pengerjaan model dilakukan menggunakan framework PyTorch dan Torchvision, sementara pengerjaan website menggunakan framework Streamlit untuk frontend dan Python, Flask, dan Google Gemini API untuk backend. Frontend di deploy di Streamlit Cloud, sementara backend dan model di host di Hugging Face Spaces.

BAB II

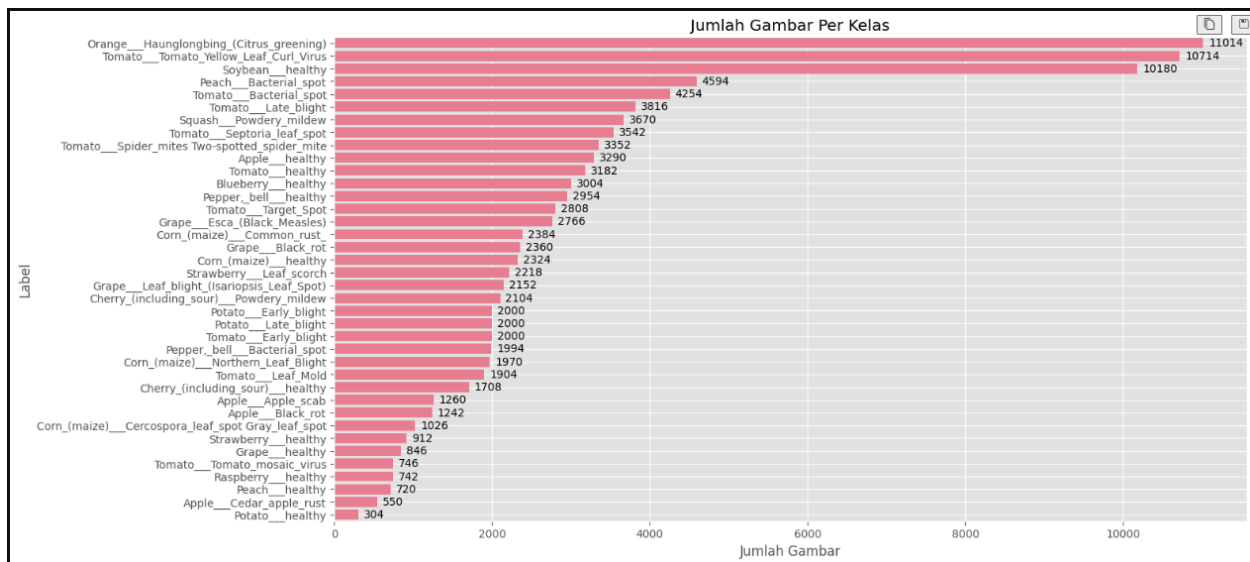
DATASET

2.1. Data Explanation

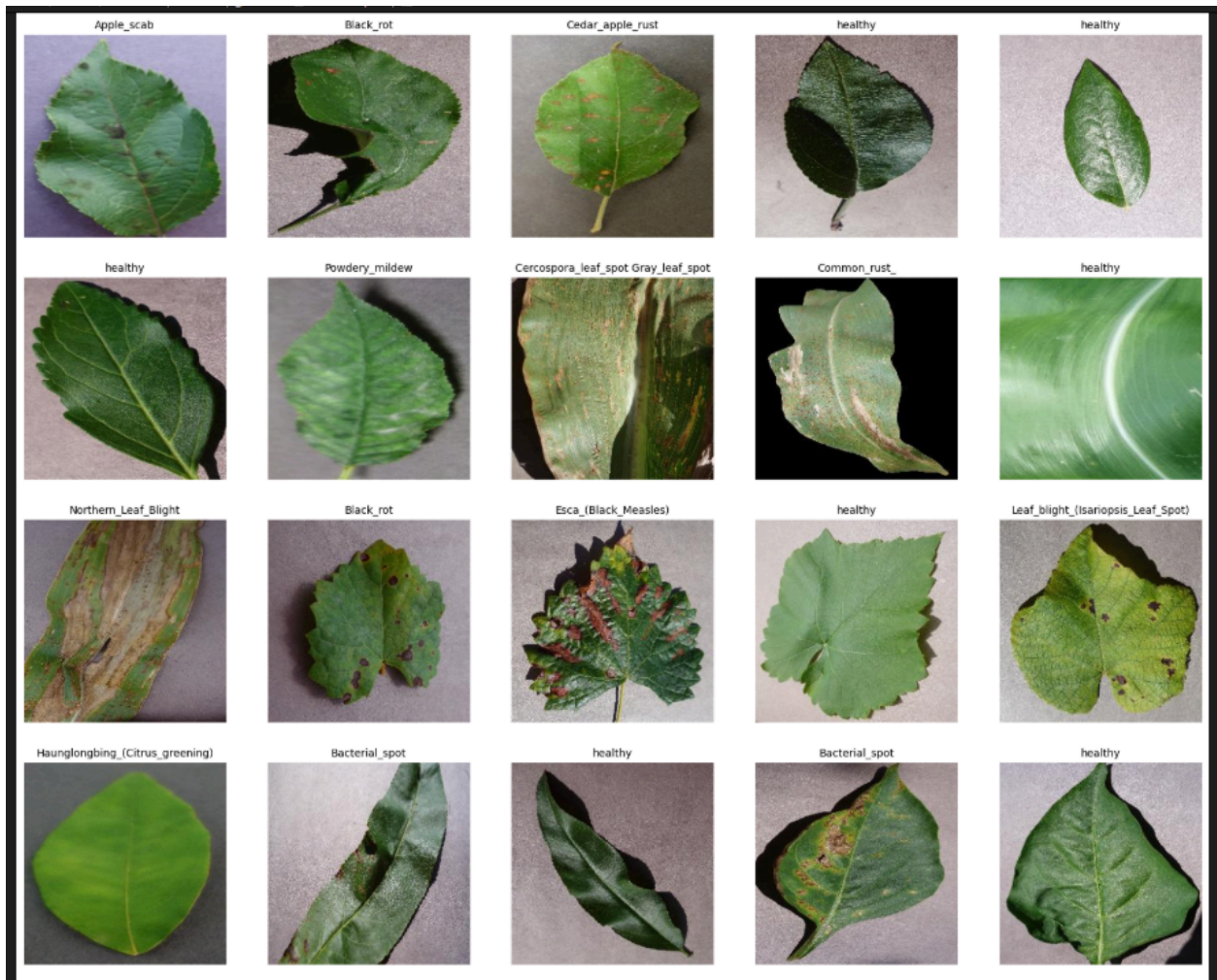
Dataset yang digunakan dalam pembuatan model ini menggunakan Plant Village Dataset. PlantVillage adalah dataset gambar daun tanaman yang digunakan untuk penelitian dan pengembangan sistem deteksi penyakit tanaman berdasarkan image. Dataset ini memiliki total jumlah gambar lebih dari 54.000 gambar yang terbagi menjadi 38 kelas kategori penyakit dan tanaman. Gambar-gambar tersebut dilabeli dengan format *jenisTanaman__StatusDaun*. Dataset ini juga memiliki 14 jenis tanaman, seperti apel, tomat, potato, kentang, dll.

Sumber: <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>

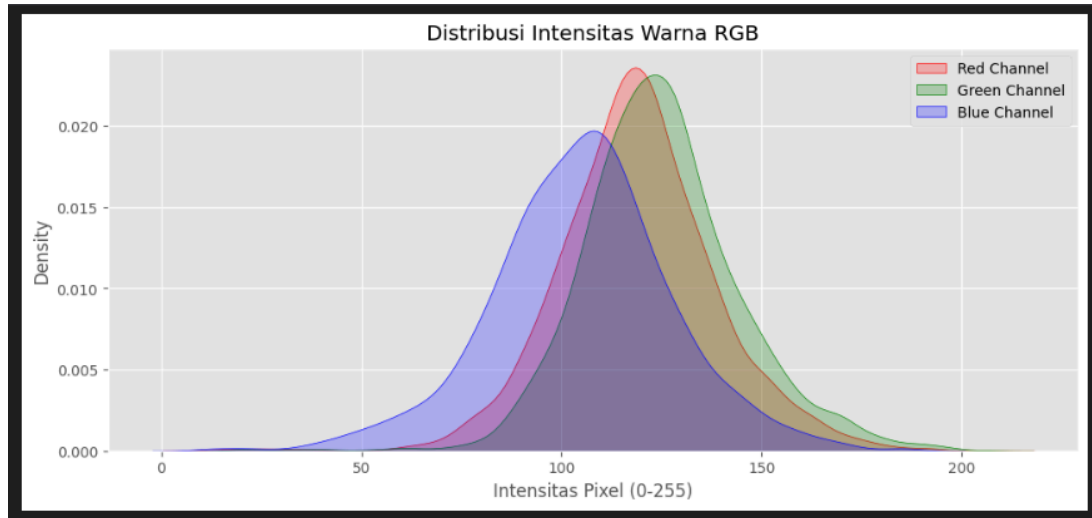
2.2. Eksplorasi (EDA)



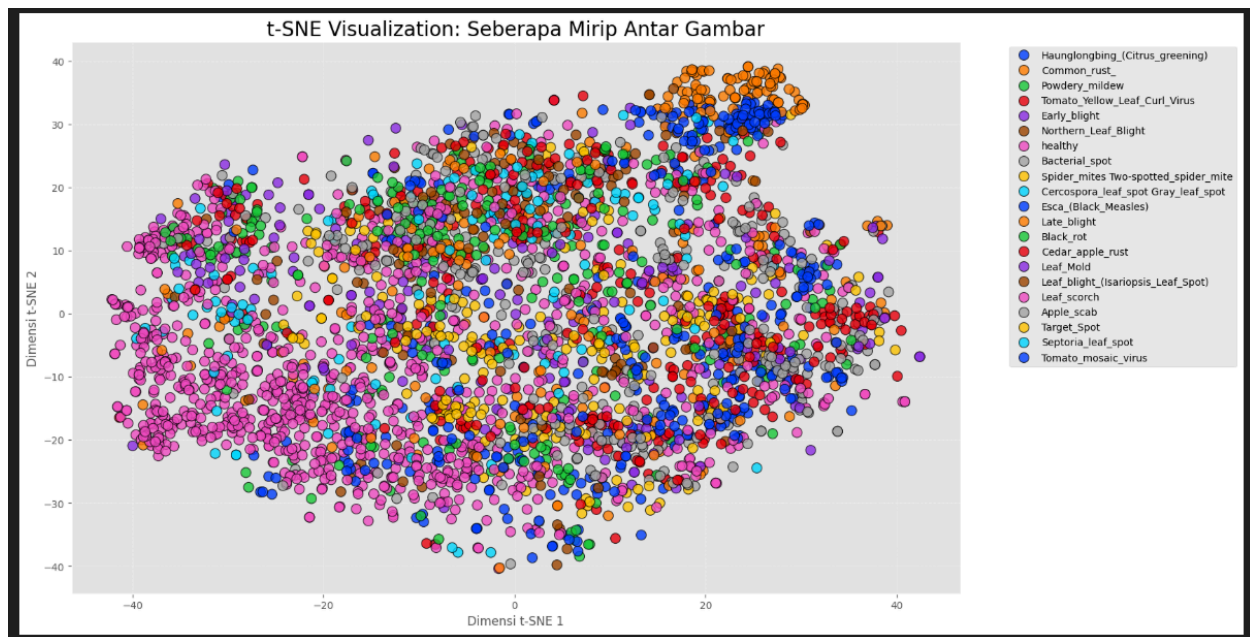
Bisa dilihat bahwa dataset PlantVillage memiliki distribusi kelas yang tidak seimbang. Beberapa kelas memiliki jumlah gambar yang sangat besar (ribuan), sedangkan kelas lainnya memiliki jumlah gambar yang jauh lebih sedikit (hanya ratusan). Ketidakseimbangan ini dapat menyebabkan model menjadi bias terhadap kelas mayoritas sehingga performa klasifikasi pada kelas minor menjadi buruk. Untuk mengatasi hal tersebut, kami menghapus beberapa class yang sekiranya memiliki image yang sedikit dan salah satunya adalah Potato__healthy.



Gambar diatas adalah gambaran umum daun dalam dataset yang tersedia.



Gambar diatas adalah distribusi intensitas warna RGB di dalam dataset kita. Bisa dilihat bahwa Red Channel memiliki density tertinggi yang menandakan bahwa mungkin di dataset banyak gambar daun yang sakit dibandingkan dengan sehat karena biasanya daun yang sakit (cacat) memiliki warna coklat kuning sedangkan daun yang sehat punya warna hijau dominan



Gambar diatas adalah t-SNE Visualization dari gambar di dataset kita. Visualisasi ini berguna untuk mengetahui seberapa mirip antara 1 gambar dengan gambar lainnya. Bisa dilihat bahwa banyak titik dengan warna yang sama berpecah (tidak kumpul di satu tempat) yang menandakan bahwa model bisa cukup sulit dalam membedakan antara 1 gambar dengan gambar lainnya (kelas-kelasnya mirip secara visual)

BAB III

MODELLING

3.1. Arsitektur Model

Pada proyek ini, kami menggunakan model Efficient Net karena menghasilkan akurasi yang lebih baik dibandingkan dari dua model yang telah kami coba dengan dataset kami, yaitu Convolutional Neural Network(CNN) dan CNN juga tetapi dengan Residual block(Resnet). Alasan lain kenapa kami menggunakan Efficient Net adalah:

- Efficient Net Menggunakan **compound scaling** yang lebih seimbang dalam mengatur kedalaman, lebar, dan resolusi model
- Jumlah parameter dan FLOPS lebih rendah
- Kebutuhan memori juga lebih rendah, sehingga proses deployment lebih mudah dilakukan

EfficientNet model merupakan sejenis model Convolutional Neural Network(CNN) yang telah menerapkan *Compound scaling*, yaitu suatu metode untuk meningkatkan performa dengan memperbesar *depth*, *width*, dan *resolution* secara bersamaan dan seimbang, sehingga lebih efisien dan akurat. Jadi penambahan metode inilah dapat membuat EfficientNet lebih unggul dari CNN biasa karena EfficientNet dapat memperbesar gambar dengan *Compound scaling* sehingga gambar tersebut lebih jelas dilihat dan fitur-fitur tersembunyi bisa terdeteksi dengan mudah.

3.2 Skenario Pelatihan

3.2.1 Dataset Splitting

- 80% untuk training
- 10% untuk validation
- 10% untuk testing

3.2.2 Data Preprocessing

1. **Normalize** → Dengan menggunakan Z-score normalization per channel(RGB)
2. **Data Augmentation** → Untuk membuat gambar terlihat berbeda-beda, ini dilakukan dengan teknik memutar, memotong, membalik, dan mengubah warna gambar supaya model tidak menghafal dan lebih pintar mengenali data baru

3.2.3 Training Configuration

- Model dilatih selama 15 epoch menggunakan GPU (jika tersedia)
- Batch size sebesar 32
- Learning rate sebesar 0.001

3.3 Diagram Arsitektur

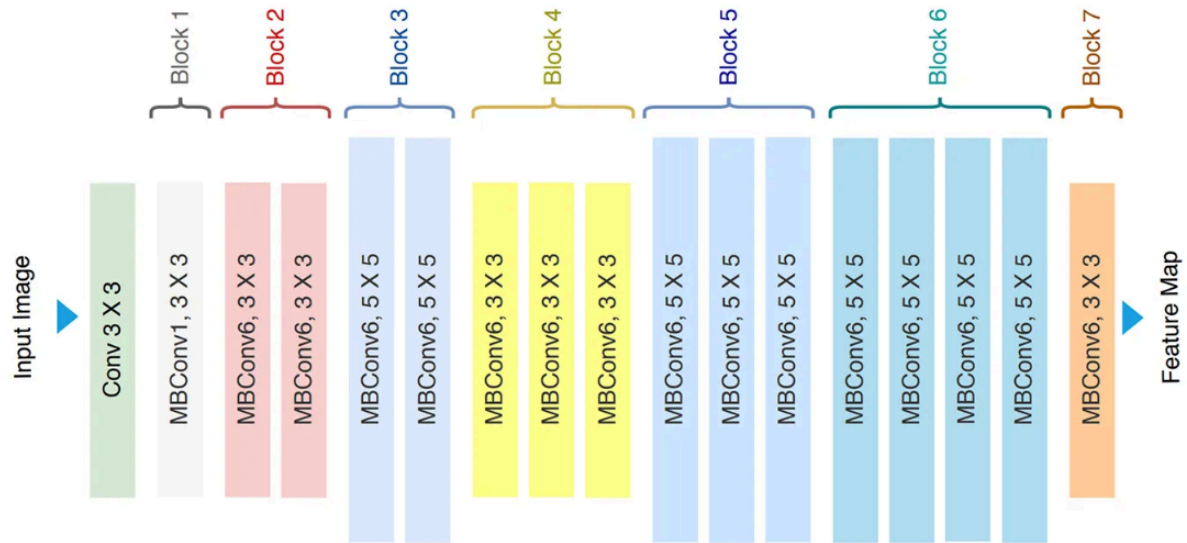


Fig 3. Architecture of EfficientNet-B0 (Source)

Efficient Net memiliki 3 jenis *layer*:

1. *Layer* input → Tempat gambar-gambar masuk ke model
2. *Layer* Conv 3 x 3 → Tempat si model mulai mengekstraksi fitur dasar gambar
3. Blok MBConv → Peta fitur yang dihasilkan di Conv 3 x 3 akan diproses disini supaya mendapatkan fitur yang lebih kompleks

BAB IV

EVALUATION

4.1 Metriks Evaluasi

Karena proyek ini merupakan proyek klasifikasi, metrik evaluasi yang digunakan adalah:

- Training Accuracy
- Training Loss
- Validation Accuracy
- Validation Loss
- Accuracy
- Precision
- Recall
- F1-Score

Metrics	Score
Training Accuracy	82.20%
Training Loss	0.62
Validation Accuracy	87.36%
Validation Loss	0.45

Model memperoleh accuracy sebesar **87.36%** pada dataset validasi, menunjukkan bahwa model cukup baik dalam mengklasifikasi sebagian besar sampel dengan benar.

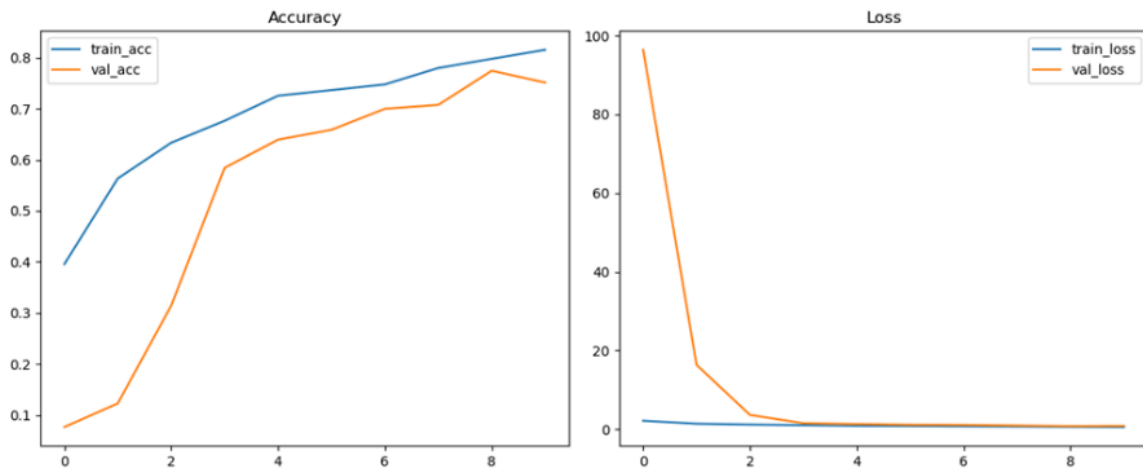
Metrics	Score
Accuracy	86%
Weighted Recall	86%
Weighted Precision	89%
Weighted F1-Score	87%

4.2 Comparison Perbandingan

4.2.1 Menggunakan CNN (ResNet)

Metrics	Score
Training Accuracy	82.56%
Training Loss	0.5531
Validation Accuracy	75.10%
Validation Loss	0.8489

Advanced CNN With Residual Block (ResNet)



Dapat dilihat dari metric bahwa Efficient Net memiliki performa yang lebih baik daripada CNN ResNet. Hal ini bisa disebabkan karena beberapa hal, yaitu:

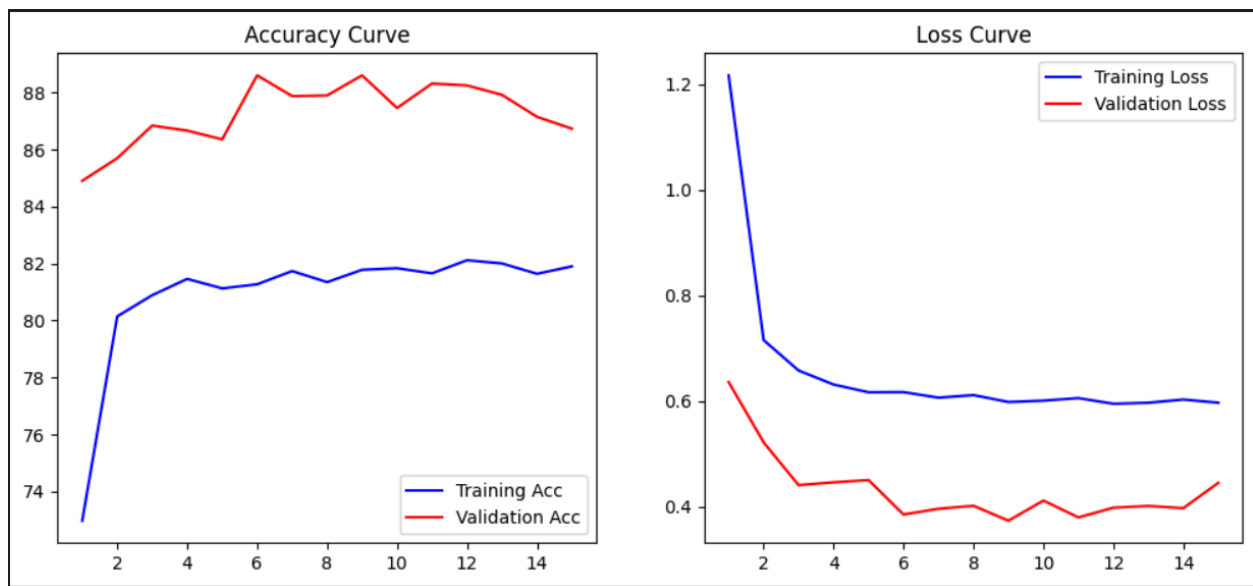
Pertama, EfficientNet menerapkan compound scaling, yaitu strategi penskalaan model yang secara seimbang mengatur kedalaman jaringan, lebar layer, dan resolusi input. Pendekatan ini memungkinkan model mengekstraksi fitur secara lebih optimal tanpa harus memperbesar model secara berlebihan, sedangkan pada CNN dengan residual block, peningkatan performa umumnya dicapai dengan menambah kedalaman jaringan.

Kedua, meskipun residual block membantu mengatasi masalah vanishing gradient dan memungkinkan jaringan menjadi lebih dalam, arsitektur CNN dengan residual block tetap memiliki jumlah parameter dan kebutuhan komputasi yang lebih besar dibandingkan EfficientNet. Pada dataset PlantVillage yang memiliki pola visual antar kelas yang cukup mirip, model yang terlalu kompleks cenderung kurang efisien dan berisiko mengalami overfitting.

Ketiga, EfficientNet yang digunakan dalam penelitian ini memanfaatkan transfer learning sehingga model telah memiliki representasi fitur visual umum yang kuat sejak awal training. Hal ini membuat proses fine-tuning menjadi lebih cepat dan stabil dibandingkan CNN dengan residual block yang dilatih dari awal atau dengan bobot yang kurang optimal.

4.3 Visualisasi Hasil

4.3.1 Accuracy and Validation Curve



Berdasarkan grafik, dapat dilihat bahwa **training loss mengalami penurunan seiring bertambahnya epoch**. Hal yang sama juga terjadi pada **validation loss**, di mana nilainya ikut menurun dan tetap berada di bawah training loss. Kondisi ini menunjukkan bahwa model **belajar dengan baik** dan **tidak mengalami overfitting**.

Hal ini juga diperkuat oleh grafik akurasi. **Akurasi training meningkat secara bertahap seiring bertambahnya epoch**, dan tren peningkatan yang sama juga terlihat pada **akurasi validation**. Meskipun nilai akurasi validation sedikit lebih tinggi dibandingkan akurasi training, perbedaannya masih dalam batas wajar dan tidak menunjukkan indikasi overfitting.

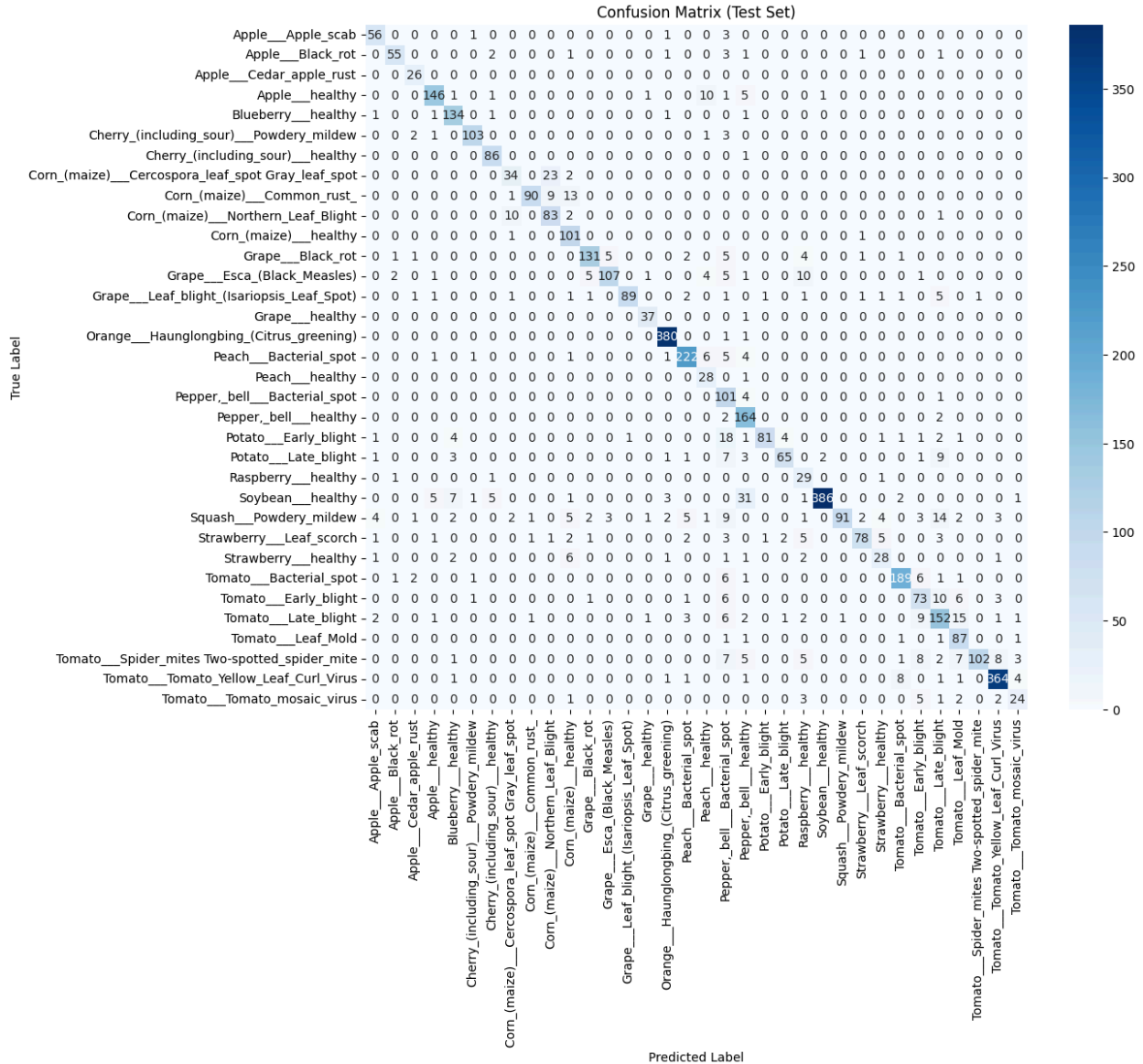
Secara keseluruhan, grafik loss dan akurasi menunjukkan bahwa model memiliki **kemampuan generalisasi yang baik** dan mampu mempelajari pola data tanpa menghafal data training.

4.3.2 Classification Report

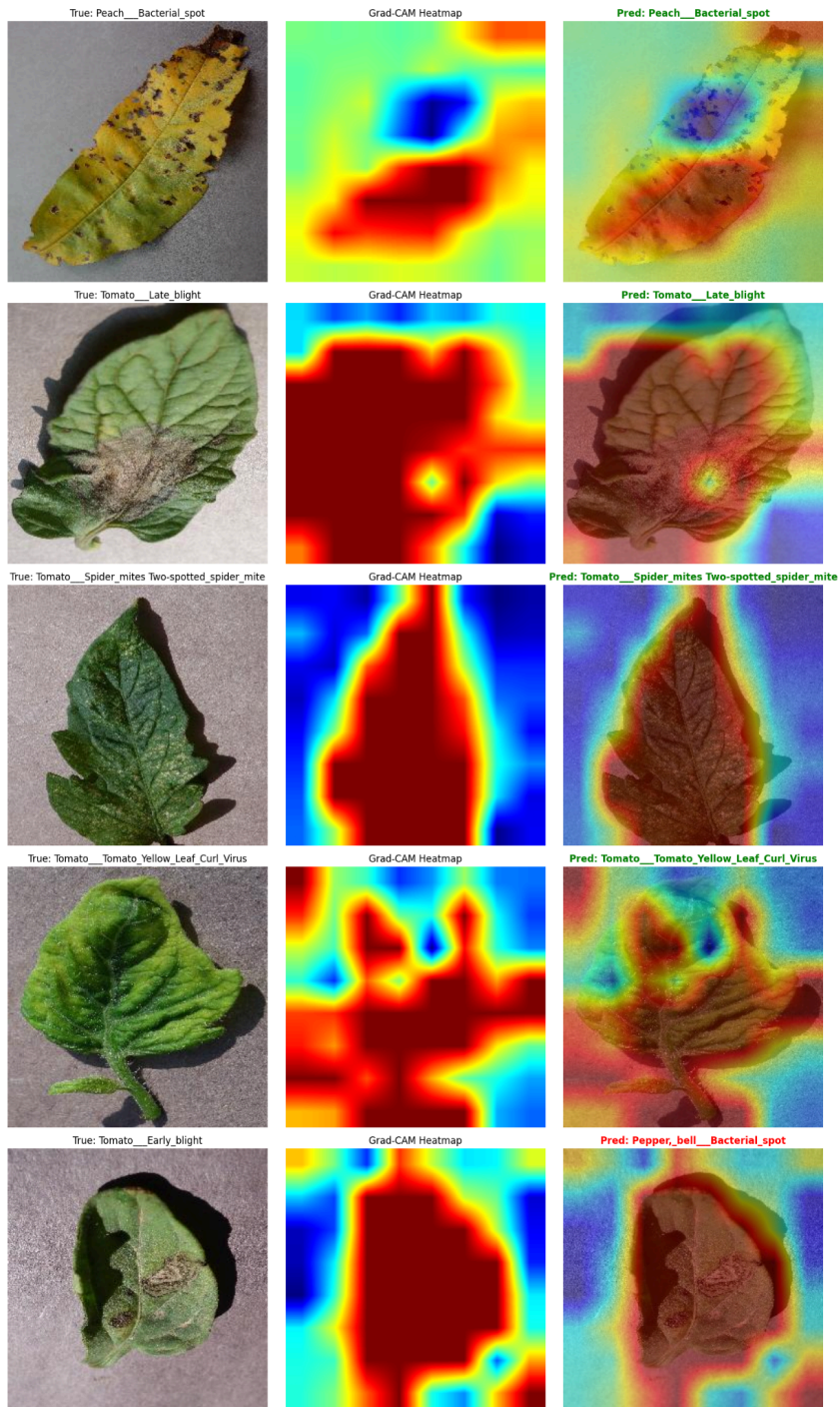
Classification Report:				
	precision	recall	f1-score	support
Apple__Apple_scab	0.84	0.92	0.88	61
Apple__Black_rot	0.92	0.85	0.88	65
Apple__Cedar_apple_rust	0.79	1.00	0.88	26
Apple__healthy	0.92	0.88	0.90	166
Blueberry__healthy	0.86	0.96	0.91	139
Cherry_(including_sour)__Powdery_mildew	0.95	0.94	0.94	110
Cherry_(including_sour)__healthy	0.90	0.99	0.94	87
Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot	0.69	0.58	0.63	59
Corn_(maize)__Common_rust_	0.97	0.80	0.87	113
Corn_(maize)__Northern_Leaf_Blight	0.72	0.86	0.78	96
Corn_(maize)__healthy	0.74	0.98	0.85	103
Grape__Black_rot	0.93	0.87	0.90	151
Grape__Esca_(Black_Measles)	0.93	0.78	0.85	137
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	0.99	0.82	0.90	108
Grape__healthy	0.90	0.97	0.94	38
Orange__Haunglongbing_(Citrus_greening)	0.97	0.99	0.98	382
Peach__Bacterial_spot	0.93	0.92	0.93	241
Peach__healthy	0.56	0.97	0.71	29
Pepper,_bell__Bacterial_spot	0.52	0.95	0.67	106
Pepper,_bell__healthy	0.71	0.98	0.82	168
Potato__Early_blight	0.98	0.70	0.81	116
Potato__Late_blight	0.90	0.70	0.79	93
Raspberry__healthy	0.46	0.91	0.61	32
Soybean__healthy	0.99	0.87	0.93	443
Squash__Powdery_mildew	0.99	0.58	0.73	158
Strawberry__Leaf_scorch	0.93	0.74	0.82	106
Strawberry__healthy	0.70	0.65	0.67	43
Tomato__Bacterial_spot	0.93	0.91	0.92	208
Tomato__Early_blight	0.68	0.72	0.70	101
Tomato__Late_blight	0.74	0.77	0.75	198
Tomato__Leaf_Mold	0.71	0.95	0.81	92
Tomato__Spider_mites Two-spotted_spider_mite	0.99	0.68	0.81	149
Tomato__Tomato_Yellow_Leaf_Curl_Virus	0.95	0.95	0.95	382
Tomato__Tomato_mosaic_virus	0.71	0.63	0.67	38
accuracy			0.86	4544
macro avg	0.84	0.85	0.83	4544
weighted avg	0.89	0.86	0.87	4544

Jika dilihat dari presisi untuk tiap kelas, mayoritas memiliki nilai presisi yang melebihi 80 persen. Ini membuktikan kalau model memiliki tingkat ketepatan yang baik dalam memprediksi kelas, di mana sebagian besar prediksi yang dihasilkan sesuai dengan kelas sebenarnya.

4.3.3 Confusion Matrix



4.3.4 GRAD-CAM



Dapat dilihat bahwa warna merah di GRAD-CAM nya hampir semuanya berada di sekitar daun yang menandakan bahwa model memprediksi berdasarkan daunnya bukan background gambarnya

BAB V

DEPLOYMENT

5.1 Aplikasi

5.1.1 Frontend (User Interface)

- **Streamlit:** Framework berbasis Python yang digunakan untuk membangun web yang interaktif. Disini streamlit akan digunakan untuk mengambil input gambar dari user dan akan diteruskan ke backend melalui API

5.1.2 Backend & Logic

- **Python:** Bahasa pemrograman inti yang menangani seluruh logika aplikasi, orkestrasi model, dan integrasi API.
- **Flask:** Framework web mikro berbasis Python yang digunakan untuk membuat REST API. Flask bertugas menerima request gambar dari frontend, menjalankan inferensi model, dan mengembalikan hasil prediksi dalam format JSON.
- **Google Gemini API:** Layanan Generative AI yang dihubungkan ke backend untuk memberikan rekomendasi penanganan penyakit dan tips perawatan tanaman berdasarkan hasil prediksi model.

5.1.3 Deep Learning

- **PyTorch:** Framework Deep Learning yang digunakan untuk melatih dan menjalankan model neural network.
- **Torchvision:** Digunakan untuk *preprocessing* dan augmentasi data citra (normalisasi tensor) agar sesuai dengan input model.

5.1.4. Hosting

Untuk memastikan aplikasi dapat diakses secara publik dengan performa komputasi yang baik untuk Deep Learning, kami menggunakan strategi deployment berikut:

- **Frontend (Interface):** Di-deploy di **Streamlit Cloud**
- **Backend & Model Hosting:** Di-deploy di **Hugging Face Spaces**.

5.2 Alur Kerja

Alur kerja aplikasi ini menjelaskan tahapan proses mulai dari pengguna mengunggah gambar daun hingga sistem menghasilkan prediksi penyakit beserta rekomendasi penanganan tanaman. Berikut adalah tahapan prosesnya:

5.2.1 Input Pengguna (Frontend)

1. Pengguna mengakses aplikasi melalui antarmuka web Streamlit yang telah dideploy di Streamlit Cloud.
2. Pengguna mengunggah gambar daun tanaman melalui komponen upload gambar yang disediakan.
3. Setelah gambar diunggah, Streamlit akan:
 - Melakukan validasi format gambar (harus memiliki ekstension image).
 - Mengirimkan gambar tersebut ke backend Flask melalui REST API dalam bentuk HTTP request (POST).

5.2.2 Pemrosesan Backend & Model

1. Flask Backend menerima request gambar dari frontend.
2. Gambar yang diterima akan diproses melalui tahap preprocessing yang meliputi:
 - Resize gambar sesuai ukuran input model.
 - Normalisasi nilai pixel.
 - Konversi gambar menjadi tensor PyTorch.
3. Model Deep Learning berbasis PyTorch yang telah dilatih sebelumnya akan dimuat ke memori (load model).
4. Tensor hasil preprocessing dimasukkan ke dalam model untuk dilakukan klasifikasi.
5. Model menghasilkan output berupa:
 - Label kelas penyakit daun.
 - Nilai confidence score atau probabilitas prediksi.

5.2.3 Integrasi Google Gemini API (Rekomendasi AI)

1. Hasil prediksi penyakit dari model akan diteruskan ke Google Gemini API.
2. Backend menyusun prompt yang berisi nama penyakit hasil klasifikasi dan informasi konteks tanaman.
3. Google Gemini API memproses prompt tersebut dan menghasilkan:
 - Rekomendasi penanganan penyakit.
 - Tips perawatan tanaman.
 - Saran pencegahan agar penyakit tidak menyebar.
4. Hasil rekomendasi dikembalikan ke backend dalam bentuk teks.

5.2.4 Output ke Pengguna

1. Backend Flask mengirimkan response ke frontend dalam format JSON, yang berisi:
 - Hasil klasifikasi penyakit daun.
 - Confidence score prediksi.
 - Rekomendasi dan tips perawatan dari Google Gemini API.
2. Streamlit menampilkan hasil tersebut kepada pengguna secara interaktif, meliputi:
 - Nama penyakit daun.
 - Tingkat kepercayaan prediksi.

- Rekomendasi dan tips perawatan tanaman yang mudah dipahami.

BAB VI

REFLECTION

6.1 Analisis Kritis

Pendekatan Deep Learning yang digunakan dalam penelitian ini terbukti mampu mengklasifikasikan penyakit tanaman berdasarkan gambar daun dengan cukup baik pada dataset PlantVillage. Namun, terdapat beberapa keterbatasan yang perlu diperhatikan terkait kesesuaian pendekatan ini terhadap kondisi penggunaan nyata.

Salah satu keterbatasan utama terletak pada cakupan datasetnya. Dataset PlantVillage tidak mencakup seluruh jenis tanaman dan penyakit yang ada di dunia nyata sehingga kemampuan generalisasi model menjadi terbatas pada kelas-kelas yang tersedia dalam dataset. Hal ini menyebabkan model belum tentu mampu mengenali penyakit atau jenis tanaman di luar data pelatihan.

Selain itu, sebagian besar gambar dalam dataset diambil dalam kondisi ideal, seperti pencahayaan yang baik, latar belakang yang bersih, serta fokus gambar yang jelas pada daun. Kondisi ini berbeda dengan situasi nyata di lapangan, di mana gambar dapat memiliki pencahayaan yang tidak merata, latar belakang yang kompleks, sudut pengambilan yang bervariasi, serta kualitas kamera yang berbeda-beda. Perbedaan kondisi ini berpotensi menurunkan performa model ketika diterapkan pada data dunia nyata.

Meskipun demikian, penggunaan arsitektur EfficientNet dengan transfer learning membantu meningkatkan kemampuan model dalam mengekstraksi fitur visual secara efisien. Namun, untuk meningkatkan performa model di masa depan, diperlukan dataset yang lebih beragam serta pengujian lebih lanjut pada data real world agar model dapat beradaptasi dengan kondisi yang lebih kompleks.

6.2 Kendala & Pembelajaran

Salah satu kendala utama yang dihadapi dalam pengembangan sistem ini adalah pengelolaan alur komunikasi antara frontend, backend, dan model Deep Learning. Aplikasi memerlukan mekanisme state management yang baik agar konteks antara input pengguna, proses inferensi model, serta hasil prediksi dapat tersampaikan dengan benar dan konsisten sehingga pengalaman pengguna tetap nyaman.

Gambar yang diunggah oleh pengguna melalui frontend harus diproses terlebih dahulu di backend agar sesuai dengan format input model. Setelah itu, hasil prediksi dari model perlu dikirim kembali ke frontend dalam bentuk yang mudah dipahami oleh pengguna. Lalu ada juga integrasi Google Gemini API, karena prompt yang dikirimkan ke Gemini harus memuat konteks penyakit hasil prediksi model agar rekomendasi yang dihasilkan tetap relevan.

Untuk mengatasi kompleksitas tersebut, solusi yang diterapkan adalah memisahkan (decoupling) frontend dan backend. Frontend dibangun menggunakan Streamlit untuk menangani interaksi pengguna dan visualisasi hasil, sedangkan backend menggunakan Flask sebagai REST API untuk menangani pemrosesan gambar, inferensi model, serta komunikasi dengan Gemini API. Model Deep Learning

dijalankan menggunakan PyTorch secara terpisah di backend sehingga arsitektur sistem menjadi lebih modular dan mudah dikembangkan.

REFERENCE

- [1] A. H. Abdalla, “PlantVillage Dataset,” *Kaggle*, 2023. [Online]. Available: <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>.
- [2] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6105–6114, 2019.
- [3] GeeksforGeeks, “EfficientNet Architecture,” 2023. [Online]. Available: <https://www.geeksforgeeks.org/computer-vision/efficientnet-architecture/>.
- [4] WisdomML, “EfficientNet and Its Performance Comparison with Other Transfer Learning Networks,” 2023. [Online]. Available: <https://wisdomml.in/efficientnet-and-its-performance-comparison-with-other-transfer-learning-networks/>.
- [5] Ultralytics, “What Is EfficientNet? A Quick Overview,” 2023. [Online]. Available: <https://www.ultralytics.com/blog/what-is-efficientnet-a-quick-overview>.
- [6] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.
- [7] Torchvision Contributors, “Torchvision: Datasets, Transforms and Models for Computer Vision,” *PyTorch*, 2023. [Online]. Available: <https://pytorch.org/vision/stable/index.html>.
- [8] Streamlit Inc., “Streamlit Documentation,” 2024. [Online]. Available: <https://docs.streamlit.io/>.
- [9] P. Ronacher, “Flask Web Framework Documentation,” 2024. [Online]. Available: <https://flask.palletsprojects.com/>.
- [10] Google, “Gemini API Documentation,” 2024. [Online]. Available: <https://ai.google.dev/>.
- [11] Geeksforgeeks, “Efficientnet Architecture,” 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-vision/efficientnet-architecture/>.
- [12] Wisdom ML, “EfficientNet and its Performance Comparison with Other Transfer Learning Networks,” 2025. [Online]. Available: <https://wisdomml.in/efficientnet-and-its-performance-comparison-with-other-transfer-learning-networks/>.
- [13] Sky Engine AI, “What is EfficientNet?,” n.d. [Online]. Available: <https://www.skyengine.ai/blog/what-is-efficientnet>
- [14] Emergent Mind, “Compound Scaling Method Overview” 2026.[Online]. Available: <https://www.emergentmind.com/topics/compound-scaling-method>

LAMPIRAN

Github:

<https://github.com/NandoG1/deep-learning-project>

Deploy:

<https://nat-vision-ai.streamlit.app/>

Demo Video:

<https://drive.google.com/file/d/1q9ZX62MS13Pmtpe--Za58lNIxttleL8c/view?usp=sharing>

Video Presentation:

https://drive.google.com/file/d/1SDEK3t1nCvTA-pWrlOkOqDDpe_FM-2u6/view?usp=sharing

PPT:

https://www.canva.com/design/DAG7TpODWx0/pZbrcmcaIVcT7Meti83DCQ/edit?utm_content=DAG7TpODWx0&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton